

Function Parameters

Actual vs Formal Parameters & Call Methods

Data Transmission • Call by Value • Call by Reference

1. The Logic of Parameters

Function parameters act as **placeholders**. They allow a function to be generic—meaning it can perform the same math or logic on different sets of data provided during execution.

Key Role: Parameters are the bridge that carries data from the `main()` function (the caller) into a specific child function (the callee).

```
void example(int x) { // x is the parameter
    // code using x
}
```

2. Actual vs Formal Parameters

This is a favorite topic in C examinations. It defines the "sender" and the "receiver."

Formal Parameters

Variables declared in the function's definition. They exist only inside the function.

```
void add(int a, int b)
```

Actual Parameters

The real values or variables passed to the function during the call.

```
add(10, 20);
```

Note: Formal parameters are also known as Dummies because they are replaced by real values at runtime.

3. Syntax and Rules

When defining parameters, you must observe three strict rules:

- **Data Type:** Every parameter must have an explicit data type.
- **Order:** The values passed must match the order of parameters defined.
- **Comma Separation:** Multiple parameters are always separated by commas.

```
void profile(char name[], int age, float height) {  
    // Parameters must be in this exact order  
}
```

4. Call by Value

In C, the default method of passing data is **Call by Value**. This means a copy of the data is sent to the function.

Crucial Fact: Changes made to formal parameters inside the function **do not** affect the actual parameters in the main program.

```
void update(int n) {  
    n = n + 10;  
}  
// main: x is 5 → call update(x) → x is still 5
```

How "Call by Value" Works

Think of it as photocopies:

1. You have an original document (Variable in Main).
2. You give a photocopy to your friend (The Function).
3. Your friend scribbles on the photocopy.
4. Your original document remains clean.

This is safer for memory but can be slower for very large data sets.

5. Call by Reference (Preview)

When we want a function to modify the original variable, we use **Call by Reference**. Instead of passing the value, we pass the **Memory Address** using pointers.

```
void update(int *n) {  
    *n = *n + 10; // Modifies original memory  
}  
// update(&x);
```

This is used for high-efficiency operations and modifying multiple variables in one function call.

6. Limits and Scope

Formal parameters are **Local** to the function. Once the function finishes its work (reaches the closing bracket), the memory and names of the formal parameters are destroyed.

Feature	Scope
Formal Parameters	Local to Function
Actual Parameters	Local to Caller (e.g., Main)

7. Common Parameter Mistakes

- **Type Mismatch:** Passing a float into an integer parameter (leads to data truncation).
- **Argument Count:** Calling a function with 2 values when it expects 3.
- **Redeclaration:** Declaring a parameter name as a local variable again inside the function.

Module Summary

You have mastered the mechanisms of data transmission via Function Parameters.

*** End of 10-Page Parameter Guide ***